

Smoothing and Time-Series Decomposition

In this session, we will explore two techniques for understanding time-series data:

1. **Smoothing**: reducing short-term fluctuations to reveal the underlying pattern.
2. **Decomposition**: separating a time series into its main components.

We will explore:

- Moving averages
- Simple Exponential Smoothing
- Holt's method
- Holt-Winters
- Additive vs. multiplicative seasonality
- Classical decomposition
- STL decomposition

The main question throughout the session is:

What structure is hidden underneath the noise in a time series?

1. Why Do We Need Smoothing?

Real-world time series often contain several sources of variation:

- **Level** — the typical value of the series
- **Trend** — long-term movement
- **Seasonality** — repeating patterns
- **Noise** — short-term random fluctuations

For example:

$$y_t = \text{structure} + \text{noise}$$

The noise can make it difficult to see the underlying structure.

Smoothing methods try to reduce the effect of short-term fluctuations so that we can better see the underlying signal.

```
In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

from statsmodels.tsa.holtwinters import (
    SimpleExpSmoothing,
    Holt,
    ExponentialSmoothing
)

from statsmodels.tsa.seasonal import (
    seasonal_decompose,
    STL
)

plt.rcParams["figure.figsize"] = (12, 5)

np.random.seed(42)

```

2. Moving Average Smoothing

A moving average replaces an observation with an average of nearby observations.

For a 3-period centered moving average:

$$MA_t = \frac{y_{t-1} + y_t + y_{t+1}}{3}$$

The idea is simple:

■ Average neighboring observations to reduce short-term fluctuations.

The size of the window controls how much smoothing occurs.

```

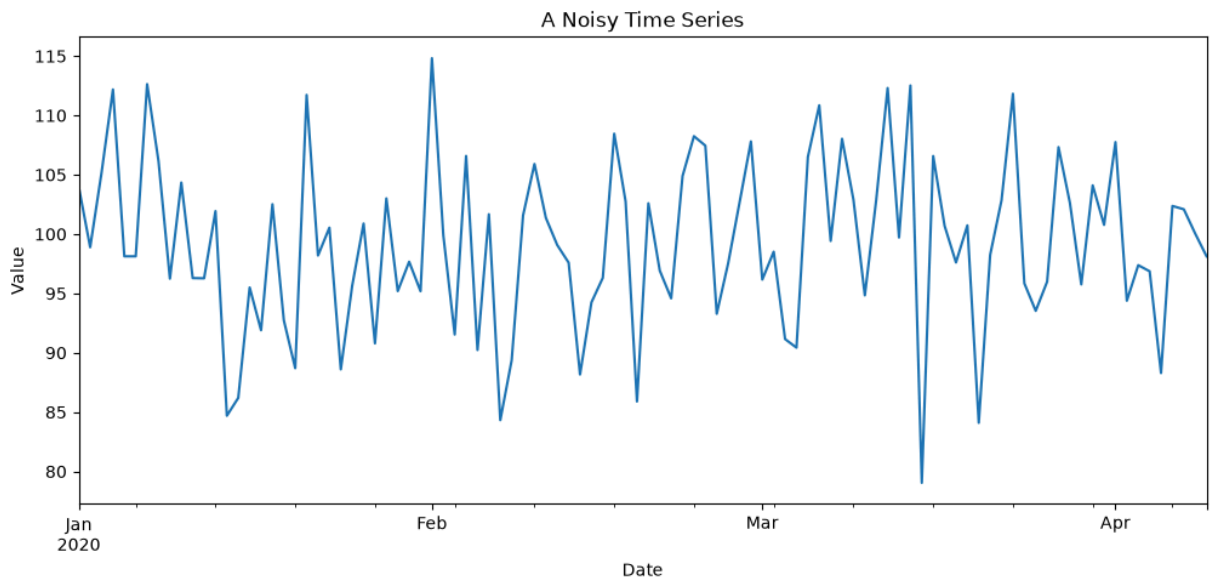
In [5]: n = 100

noise_series = pd.Series(
    100 + np.random.normal(0, 8, n),
    index=pd.date_range(
        "2020-01-01",
        periods=n,
        freq="D"
    )
)

noise_series.plot()

plt.title("A Noisy Time Series")
plt.xlabel("Date")
plt.ylabel("Value")
plt.show()

```



What do you notice?

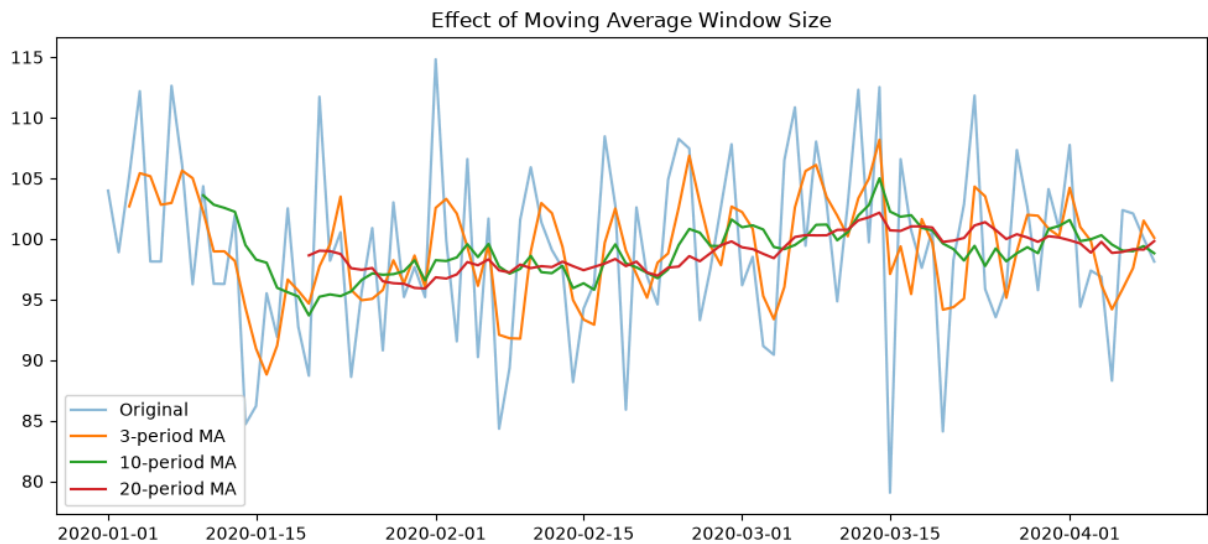
The underlying level is approximately constant, but individual observations fluctuate considerably.

Let's see whether a moving average can make the underlying level easier to see.

```
In [6]: ma_3 = noise_series.rolling(3).mean()
ma_10 = noise_series.rolling(10).mean()
ma_20 = noise_series.rolling(20).mean()

plt.plot(noise_series, label="Original", alpha=0.5)
plt.plot(ma_3, label="3-period MA")
plt.plot(ma_10, label="10-period MA")
plt.plot(ma_20, label="20-period MA")

plt.title("Effect of Moving Average Window Size")
plt.legend()
plt.show()
```



Effect of the window size

A small window:

- reacts quickly to changes
- removes less noise
- produces less smoothing

A large window:

- produces a smoother curve
- removes more short-term variation
- reacts more slowly to changes

Therefore:

larger window $\Rightarrow$ more smoothing

But more smoothing is not always better.

If the window is too large, important changes can be hidden.

3. Exponential Smoothing

Moving averages give equal weights to observations within the window. Exponential smoothing works differently. It gives more weight to recent observations and progressively less weight to older observations.

The basic idea is:

recent observations are more important

The simplest form is **Simple Exponential Smoothing**.

Simple Exponential Smoothing

Simple Exponential Smoothing models the **level** of a time series.

The updating equation is:

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}$$

where:

- ℓ_t = estimated level
- y_t = current observation
- α = smoothing parameter

with:

$$0 < \alpha < 1$$

A large α gives more weight to recent observations.

A small α produces a smoother, slower-moving estimate.

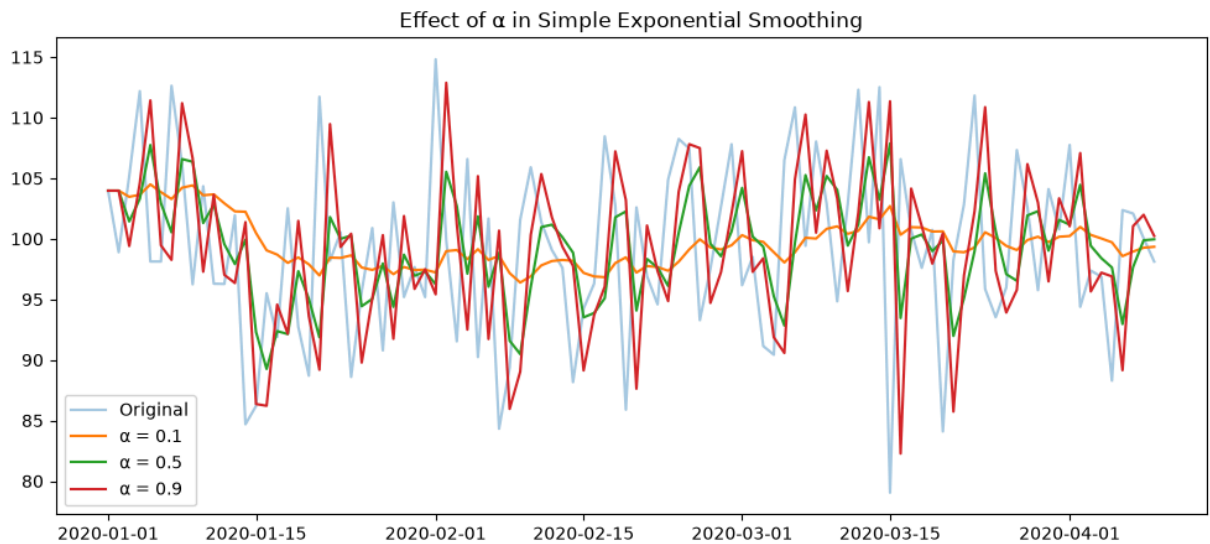
```
In [7]: alphas = [0.1, 0.5, 0.9]

plt.plot(noise_series, label="Original", alpha=0.4)

for alpha in alphas:
    model = SimpleExpSmoothing(noise_series).fit(
        smoothing_level=alpha,
        optimized=False
    )

    plt.plot(
        model.fittedvalues,
        label=f"α = {alpha}"
    )

plt.title("Effect of α in Simple Exponential Smoothing")
plt.legend()
plt.show()
```



Think about what you observed

As α increases, the smoothed series responds more strongly to recent observations.

Therefore:

 $\alpha \uparrow \Rightarrow \text{more responsive}$

while:

 $\alpha \downarrow \Rightarrow \text{smoother and slower}$

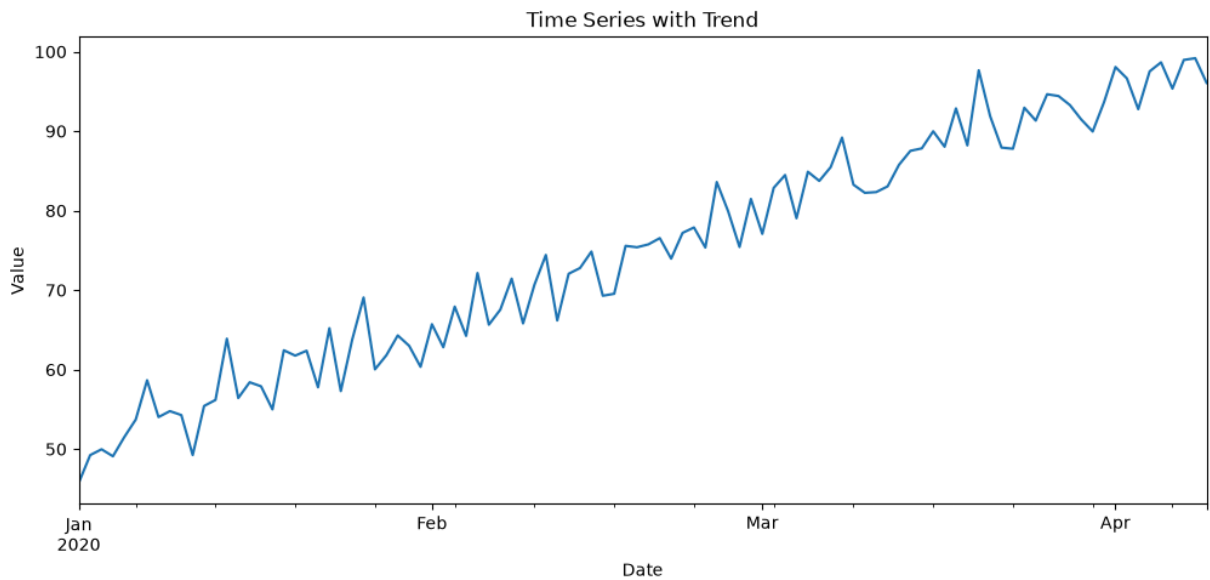
In practice, α can be estimated from the data rather than chosen manually.

```
In [8]: n = 100

trend_series = pd.Series(
    50 + 0.5 * np.arange(n) + np.random.normal(0, 3, n),
    index=pd.date_range(
        "2020-01-01",
        periods=n,
        freq="D"
    )
)

trend_series.plot()

plt.title("Time Series with Trend")
plt.xlabel("Date")
plt.ylabel("Value")
plt.show()
```



4. Holt's Method

Simple Exponential Smoothing models the level, but it does not explicitly model a trend.

When a series contains a trend, we can use **Holt's method**.

Holt's method maintains two components:

$$\boxed{\text{Level} + \text{Trend}}$$

The level is updated using:

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1})$$

The trend is updated using:

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}$$

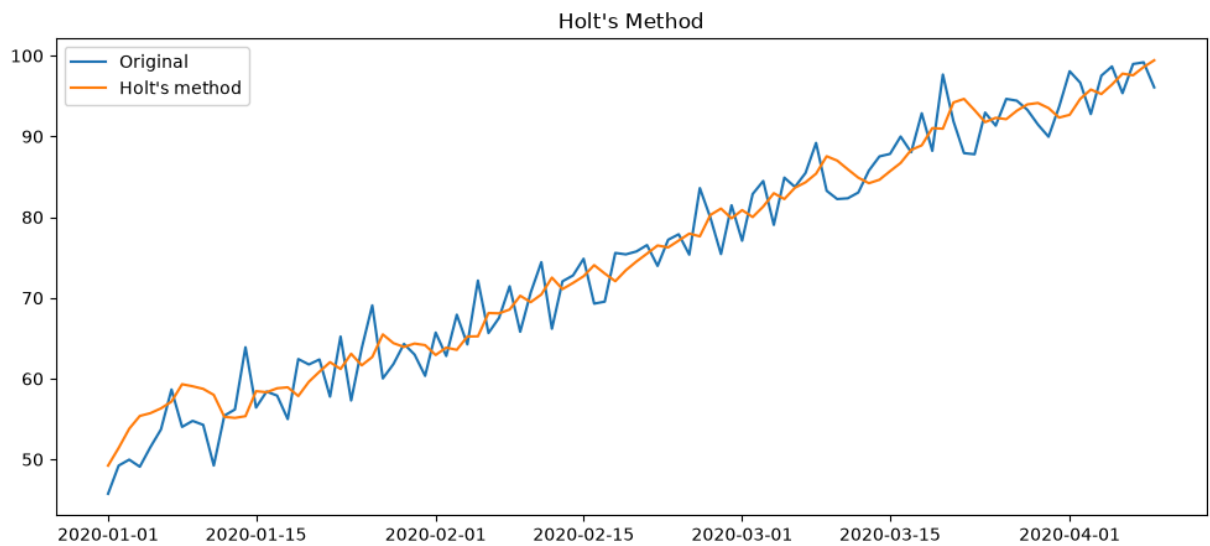
where:

- ℓ_t = level
- b_t = trend
- α = level smoothing parameter
- β = trend smoothing parameter

```
In [9]: holt_model = Holt(trend_series).fit()

plt.plot(trend_series, label="Original")
plt.plot(
    holt_model.fittedvalues,
    label="Holt's method"
)
```

```
plt.title("Holt's Method")
plt.legend()
plt.show()
```



When should we use Holt's method?

Holt's method is appropriate when the series has:

- a level
- a trend
- no strong seasonal pattern

Conceptually:

Simple Exponential Smoothing = Level

while:

Holt = Level + Trend

```
In [10]: n = 120
t = np.arange(n)

trend = 0.15 * t

seasonal_pattern = np.array([
    -10, -5, 0, 5,
    10, 15, 20, 18,
    10, 5, -3, -10
])

seasonality = np.tile(
    seasonal_pattern,
    n // 12
)
```

```

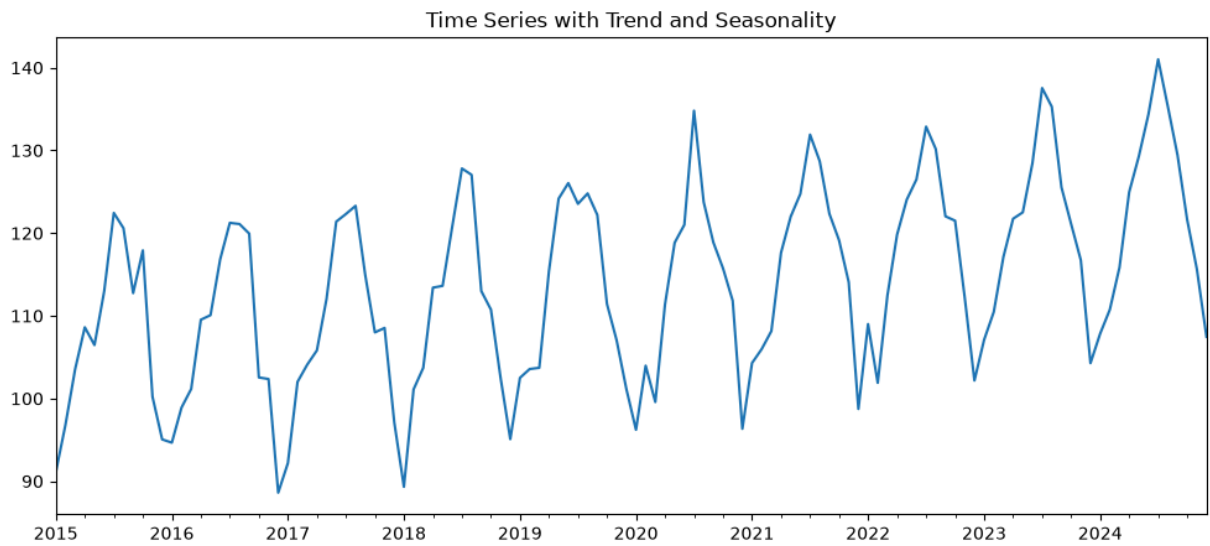
noise = np.random.normal(0, 3, n)

seasonal_series = pd.Series(
    100 + trend + seasonality + noise,
    index=pd.date_range(
        "2015-01-01",
        periods=n,
        freq="MS"
    )
)

seasonal_series.plot()

plt.title("Time Series with Trend and Seasonality")
plt.show()

```



5. Holt-Winters

Our new series contains:

- level
- trend
- seasonality

We therefore need a method that can model all three.

This is **Holt-Winters**, also called **triple exponential smoothing**.

Conceptually:

$$\text{Holt-Winters} = \text{Level} + \text{Trend} + \text{Seasonality}$$

For monthly data with yearly seasonality:

$$s = 12$$

Additive vs. Multiplicative Seasonality

There are two common ways to model seasonality.

Additive seasonality

The seasonal effect has approximately constant magnitude:

$$y_t = T_t + S_t + R_t$$

For example, the seasonal effect might be approximately:

$$+10, +20, -5, -15, \dots$$

regardless of the overall level.

Multiplicative seasonality

The seasonal effect changes with the level:

$$y_t = T_t S_t R_t$$

For example, if the level doubles, the seasonal fluctuations may also approximately double.

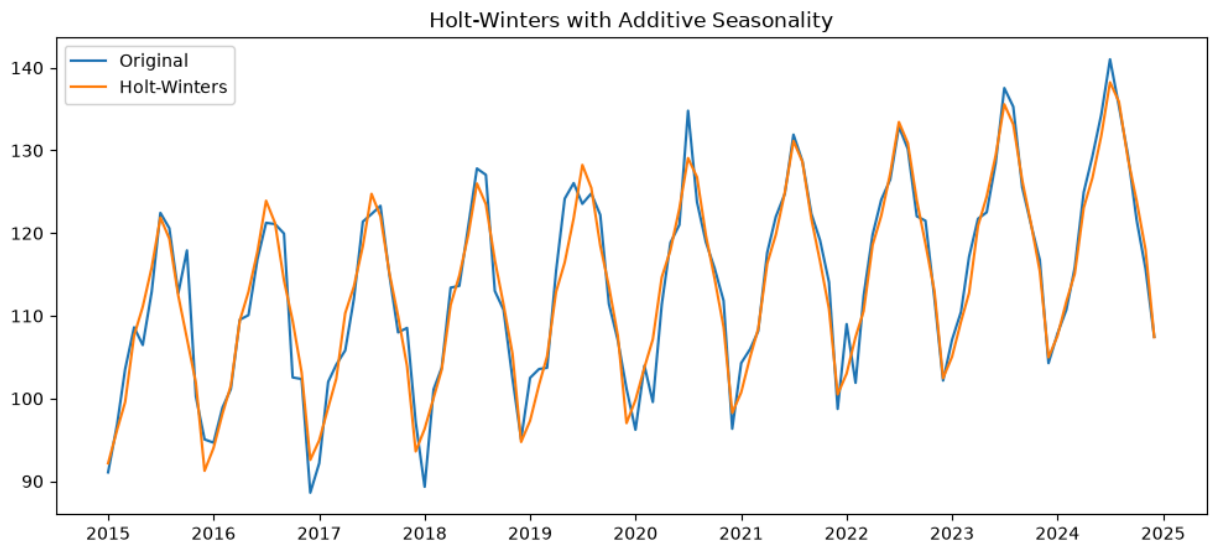
A useful visual clue:

If seasonal fluctuations become larger as the level increases, multiplicative seasonality may be more appropriate.

```
In [11]: hw_additive = ExponentialSmoothing(
    seasonal_series,
    trend="add",
    seasonal="add",
    seasonal_periods=12
).fit()

plt.plot(seasonal_series, label="Original")
plt.plot(
    hw_additive.fittedvalues,
    label="Holt-Winters"
)

plt.title("Holt-Winters with Additive Seasonality")
plt.legend()
plt.show()
```



Holt-Winters as a forecasting method

Holt-Winters does not only smooth the existing data.

It can also forecast future observations while preserving the estimated:

- level
- trend
- seasonal pattern

```
In [12]: forecast = hw_additive.forecast(12)

plt.plot(
    seasonal_series,
    label="Observed"
```

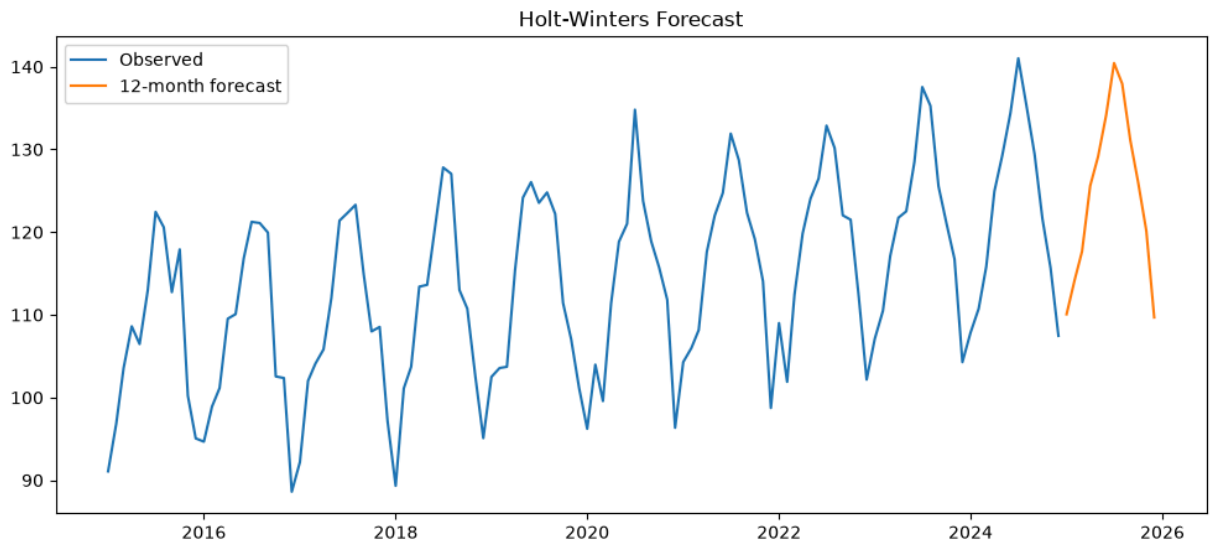
```

)

plt.plot(
    forecast,
    label="12-month forecast"
)

plt.title("Holt-Winters Forecast")
plt.legend()
plt.show()

```



6. Time-Series Decomposition

Smoothing tries to estimate the underlying signal.

Decomposition asks a different question:

What components make up the time series?

For an additive decomposition:

$$y_t = T_t + S_t + R_t$$

where:

- T_t = trend
- S_t = seasonal component
- R_t = remainder

For a multiplicative decomposition:

$$y_t = T_t S_t R_t$$

The goal is to separate the observed series into interpretable components.

7. Classical Decomposition

Classical decomposition typically estimates:

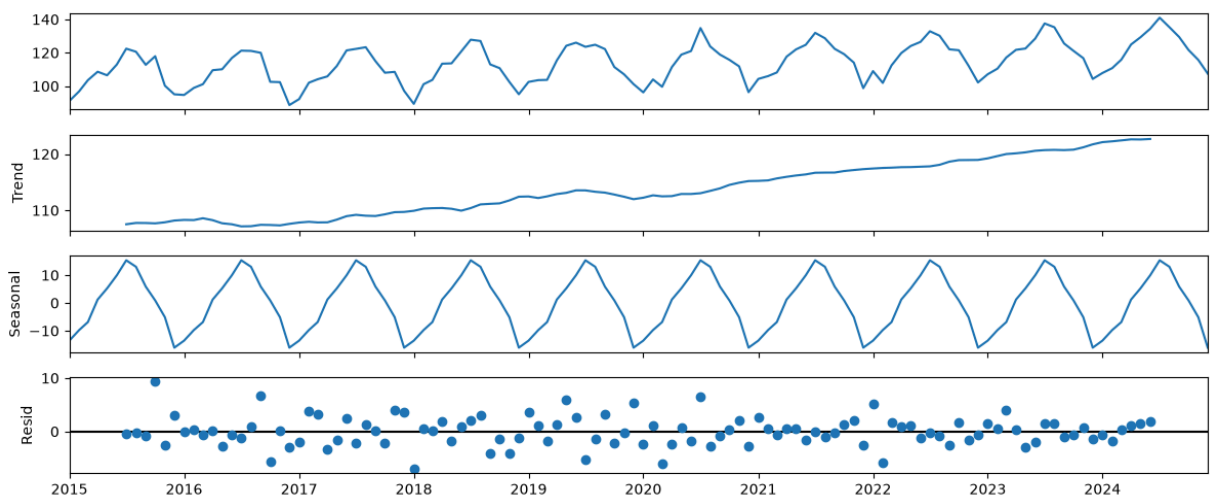
1. The trend using moving averages.
2. The seasonal component from the detrended data.
3. The remainder after removing trend and seasonality.

For additive decomposition:

$$R_t = y_t - T_t - S_t$$

The result allows us to examine each component separately.

```
In [13]: decomposition = seasonal_decompose(  
    seasonal_series,  
    model="additive",  
    period=12  
)  
  
decomposition.plot()  
  
plt.show()
```



Interpreting the decomposition

The decomposition gives us four plots:

Observed

The original time series.

Trend

The long-term movement.

Seasonal

The repeating pattern.

Remainder

What remains after removing the estimated trend and seasonal components.

For a good decomposition, the remainder should contain relatively little systematic structure.

```
In [14]: trend = decomposition.trend
seasonal = decomposition.seasonal
remainder = decomposition.resid

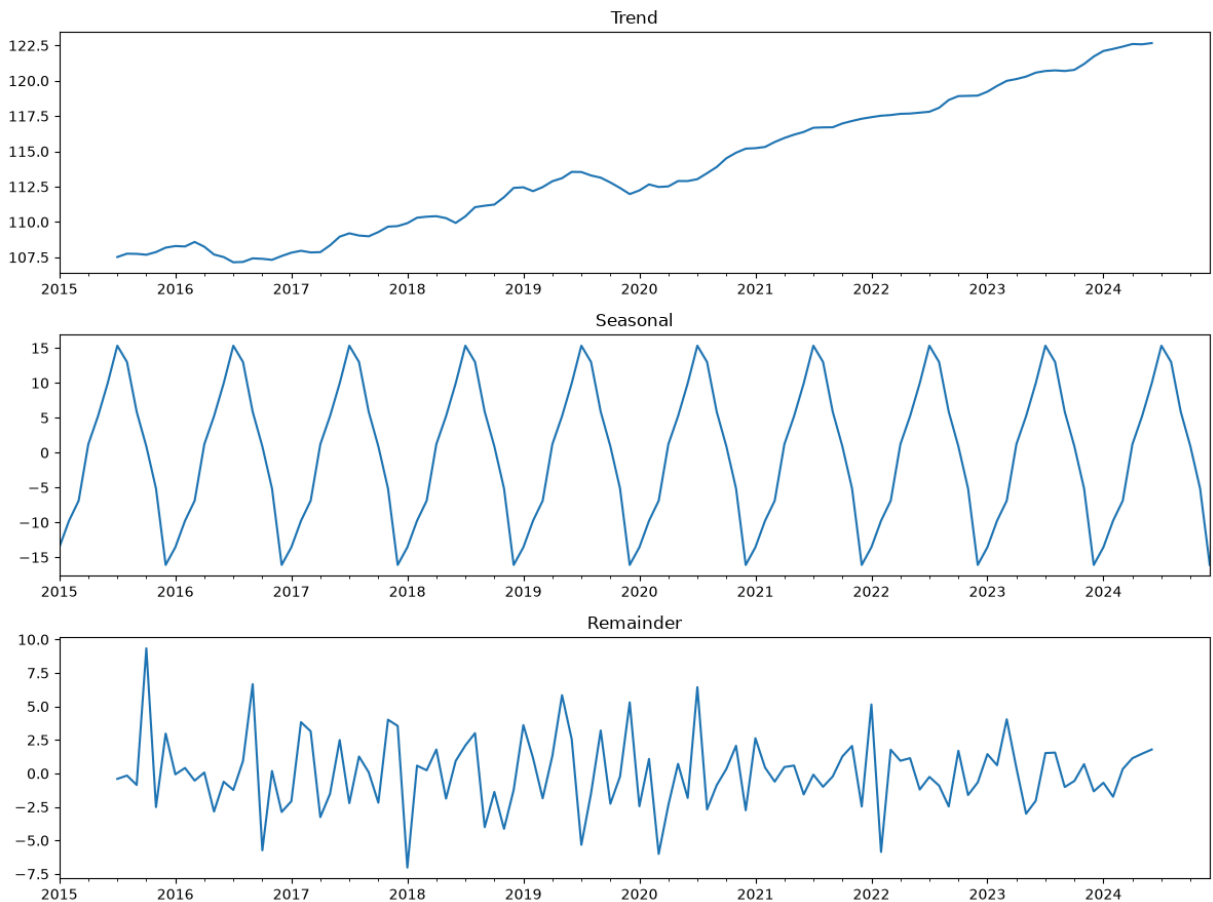
fig, axes = plt.subplots(3, 1, figsize=(12, 9))

trend.plot(ax=axes[0])
axes[0].set_title("Trend")

seasonal.plot(ax=axes[1])
axes[1].set_title("Seasonal")

remainder.plot(ax=axes[2])
axes[2].set_title("Remainder")

plt.tight_layout()
plt.show()
```



8. Additive vs. Multiplicative Decomposition

Let's compare the two assumptions.

Additive

$$y_t = T_t + S_t + R_t$$

The seasonal variation has approximately constant magnitude.

Multiplicative

$$y_t = T_t S_t R_t$$

The seasonal variation is proportional to the level.

A useful diagnostic question is:

Does the size of the seasonal fluctuations stay approximately constant, or does it grow with the level of the series?

```
In [22]: n = 120
t = np.arange(n)

# Strong upward trend
trend = 50 + 2 * t

seasonal_factors = np.array([
    0.75, 0.80, 0.90, 1.00,
    1.10, 1.25, 1.40, 1.30,
    1.15, 1.00, 0.85, 0.75
])

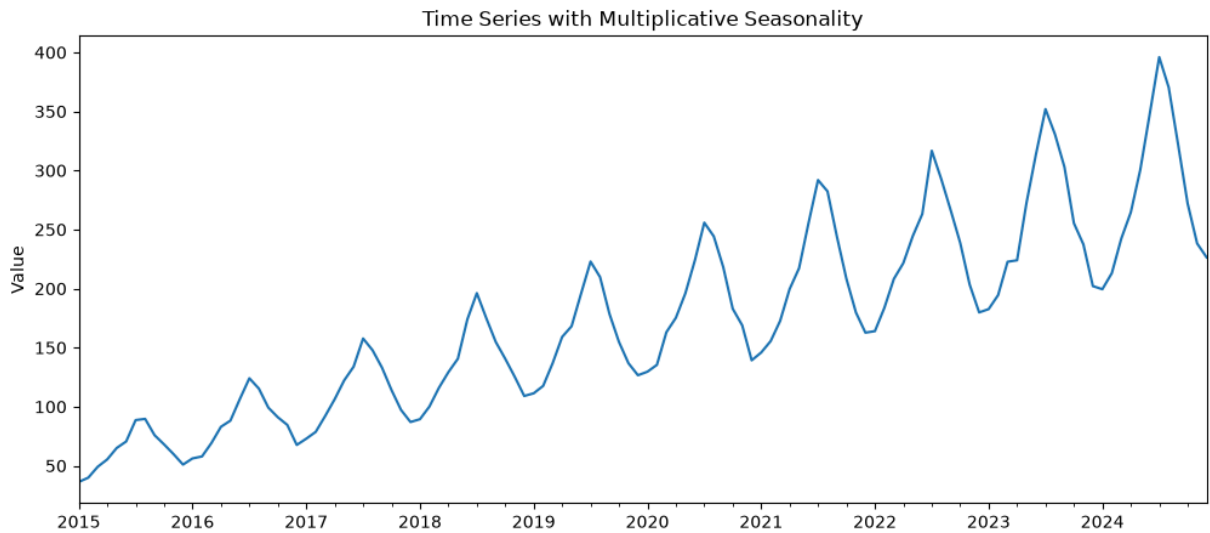
seasonality = np.tile(
    seasonal_factors,
    n // 12
)

noise = np.random.normal(1, 0.03, n)

multiplicative_series = pd.Series(
    trend * seasonality * noise,
    index=pd.date_range(
        "2015-01-01",
        periods=n,
        freq="MS"
    )
)
```

```
multiplicative_series.plot()

plt.title("Time Series with Multiplicative Seasonality")
plt.ylabel("Value")
plt.show()
```



Important interpretation

The decomposition model is an assumption about how the components combine.

Additive:

Effect of seasonality is roughly constant

Multiplicative:

Effect of seasonality depends on the level

In practice, plotting the data is often the first clue about which assumption makes more sense.

9. Examining the Remainder

After decomposition, we are left with the remainder:

$$R_t = y_t - T_t - S_t$$

For a good decomposition, the remainder should ideally have:

- mean approximately zero
- relatively constant variance
- no obvious trend
- no obvious seasonality

However, the remainder may still contain autocorrelation.

This is an important observation:

Decomposition does not necessarily explain all of the temporal dependence in a time series.

What would we like to see?

Ideally, the remainder should look approximately like random noise rather than showing:

- a trend
- repeating cycles
- long runs above/below zero
- obvious patterns

If systematic temporal structure remains, we may need a model that explicitly accounts for autocorrelation.

We will return to this idea when we study SARIMA.

10. Smoothing vs. Decomposition

Although smoothing and decomposition are related, they have different goals.

Smoothing

The main goal is to estimate a smoother version of the series.

Examples:

- Moving Average
- Simple Exponential Smoothing
- Holt
- Holt-Winters

Decomposition

The goal is to explicitly separate the series into components:

$$y_t = \text{Trend} + \text{Seasonality} + \text{Remainder}$$

or:

$$y_t = \text{Trend} \times \text{Seasonality} \times \text{Remainder}$$

Smoothing can be used as part of decomposition, but the two ideas are not the same.

11. Mini Case Study

Let's apply what we learned to a new time series.

Our goal is not to build a complicated forecasting model.

Instead, we want to answer:

1. Is there a trend?
2. Is there seasonality?
3. Is the seasonal effect additive or multiplicative?
4. Can smoothing help us see the underlying pattern?
5. What does decomposition tell us?
6. What remains unexplained after decomposition?

```
In [19]: np.random.seed(123)

n = 96
t = np.arange(n)

trend = 80 + 0.3 * t

seasonal_pattern = np.array([
    0.85, 0.90, 0.95, 1.00,
    1.05, 1.15, 1.25, 1.20,
    1.10, 1.00, 0.90, 0.85
])

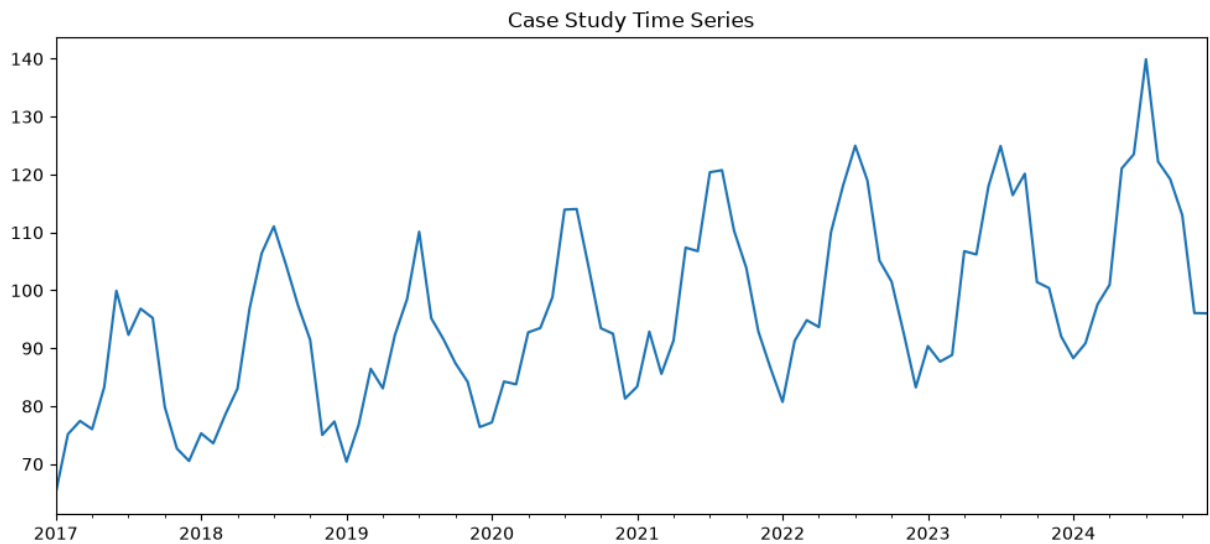
seasonality = np.tile(
    seasonal_pattern,
    n // 12
)

noise = np.random.normal(1, 0.04, n)

case_series = pd.Series(
    trend * seasonality * noise,
    index=pd.date_range(
        "2017-01-01",
        periods=n,
        freq="MS"
    )
)

case_series.plot()

plt.title("Case Study Time Series")
plt.show()
```



Before applying any method, inspect the plot and answer:

- Does the series have a trend?
- Does it have seasonality?
- Does the size of the seasonal fluctuations appear constant?
- Would you expect additive or multiplicative seasonality?

Make your hypothesis before running the next cells.

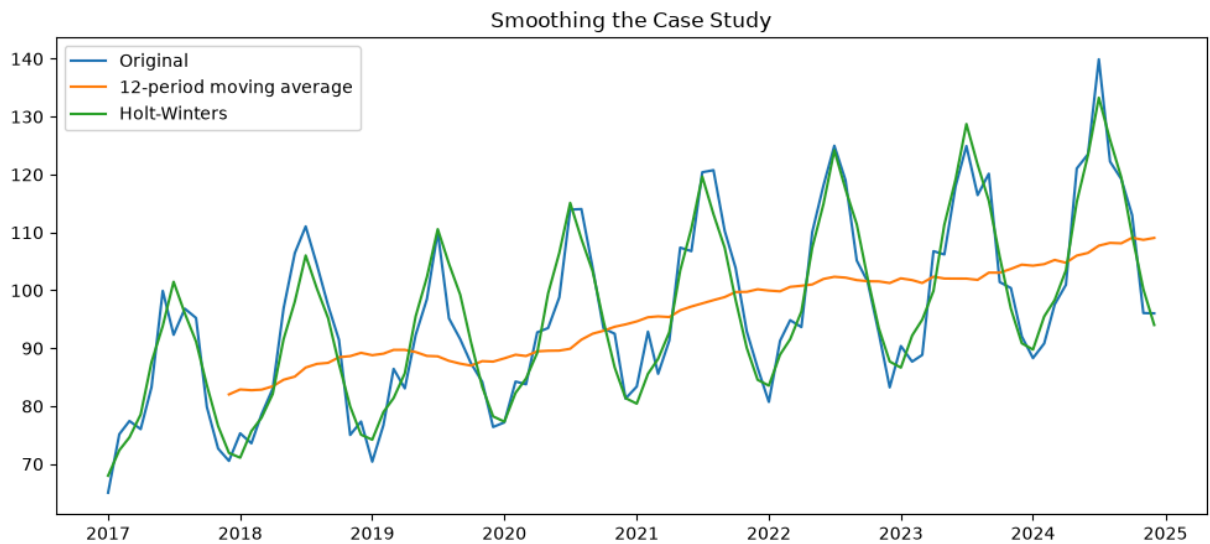
```
In [20]: # Try different smoothing methods

ma = case_series.rolling(12).mean()

hw = ExponentialSmoothing(
    case_series,
    trend="add",
    seasonal="mul",
    seasonal_periods=12
).fit()

plt.plot(case_series, label="Original")
plt.plot(ma, label="12-period moving average")
plt.plot(hw.fittedvalues, label="Holt-Winters")

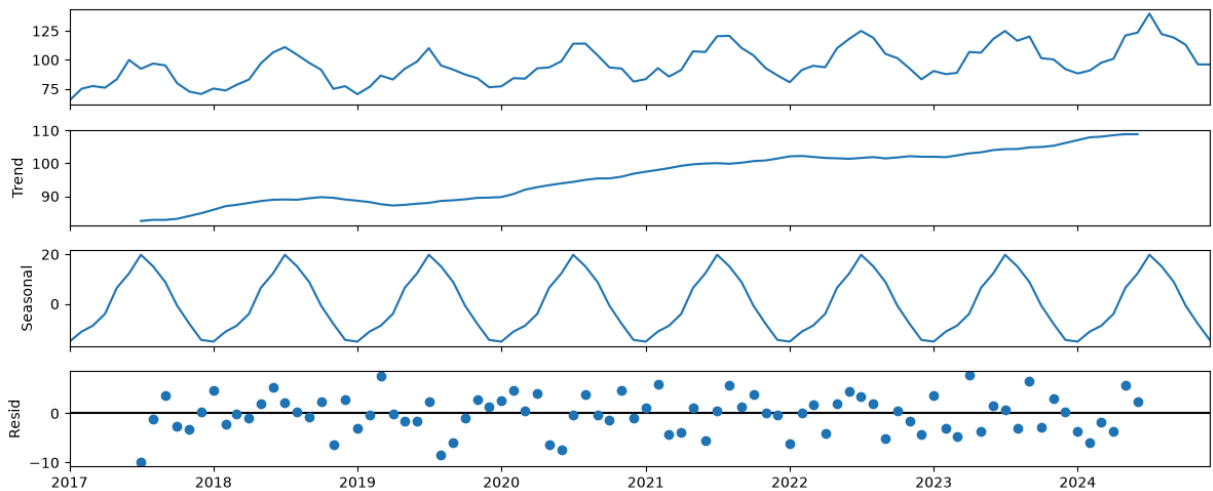
plt.title("Smoothing the Case Study")
plt.legend()
plt.show()
```



```
In [24]: decomposition = seasonal_decompose(
    case_series,
    model="additive",
    period=12
)

decomposition.plot()

plt.show()
```



Interpretation

Based on the analysis:

- The moving average helps reveal the long-term movement.
- Holt-Winters accounts for level, trend, and seasonality simultaneously.
- The remainder tells us what is left after accounting for the main structure.

The next question is:

Does the remainder still contain temporal dependence?

If it does, smoothing and decomposition alone may not be enough.

This leads naturally to our next session on **SARIMA**.

In []: